Table des matières

Cours 10 — Les codes 1

Cours 10 — Les codes

Cours 10/12 · La Brûlerie — Formation Data Engineering

Sources	Ingestion	RAW
d'où vient la donnée	aller la chercher, et la charger	la déposer, la ranger, la traiter
		✓ ici

À ouvrir après avoir essayé. Le code garde les commentaires numérotés du cours, pour comparer ligne par ligne. Le script se crée dans la première fenêtre, par le Bloc-notes ; les requêtes se tapent dans le notebook de DuckDB (`duckdb -ui`, lancé depuis `Projet_Brulerie`), une par cellule.

01_de/iceberg.py

```
# 01_de/iceberg.py : les tickets de la caisse dans une table Iceberg, un instantané par export
from pathlib import Path
import duckdb
from pyiceberg.catalog.sql import SqlCatalog

COLONNES = {"ticket_id": "INTEGER", "horodatage": "TIMESTAMP", "type_ticket": "VARCHAR", "statut": "VARCHAR",
            "ticket_origine": "INTEGER", "membre_id": "VARCHAR", "sous_total": "DECIMAL(10,2)",
            "tps": "DECIMAL(10,2)", "tvq": "DECIMAL(10,2)", "total": "DECIMAL(10,2)",
            "mode_paiement": "VARCHAR", "arrondi": "DECIMAL(10,2)", "encaisse": "DECIMAL(10,2)"}
# 1. le catalogue SQL, une base SQLite, et les fichiers de la table : l'un et l'autre dans le lac
catalogue = SqlCatalog("brulerie", uri="sqlite:///plateforme/lac/iceberg/catalogue.db",
                       warehouse=Path("plateforme/lac/iceberg").resolve().as_posix())
catalogue.create_namespace_if_not_exists("bronze")

for fichier in sorted(Path("sources/caisse").glob("*/tickets_*.csv")):
    # 2. l'export, typé comme au bronze, avec son chemin et son jour livré
    jour = fichier.stem.split("_")[1]
    tickets = duckdb.read_csv(str(fichier), dtype=COLONNES, filename=True).select(
        f"*, DATE '{jour}' AS jour").to_arrow_table()
    # 3. la table naît au premier export, partitionnée par jour, comme le bronze
    if not catalogue.table_exists("bronze.tickets"):
        table = catalogue.create_table("bronze.tickets", schema=tickets.schema)
```


© 2026 iAlexMG. Tous droits réservés.

```
        table.update_spec().add_identity("jour").commit()
    table = catalogue.load_table("bronze.tickets")
    # 4. sauter un export que l'historique de la table connaît déjà
    if fichier.name in [s.summary.get("export") for s in table.snapshots()]:
        continue
    # 5. l'export écrase son jour : le dernier export d'un jour fait foi
    table.dynamic_partition_overwrite(tickets, snapshot_properties={"export": fichier.name})
    print(f"{fichier.name} : {len(tickets)} tickets, {len(table.snapshots())} instantanés")
```

4.4 Les fichiers que la table a écrits

```
-- 1. les fichiers de données sur le disque, ceux que la table cite, puis ses fichiers de métadonnées
SELECT (SELECT COUNT(*) FROM glob('plateforme/lac/iceberg/bronze/tickets/data/*/*.parquet')) AS donnees,
       (SELECT COUNT(*) FROM iceberg_metadata(GETVARIABLE('tickets'))) AS cites,
       (SELECT COUNT(*) FROM glob('plateforme/lac/iceberg/bronze/tickets/metadata/*.metadata.json')) AS metadonnees;
```

5.2 Le 12 juin à trois instantanés

```
-- 1. le 12 juin au 12e instantané, au 13e, puis dans la table courante
SELECT (SELECT COUNT(*) FROM iceberg_scan(GETVARIABLE('tickets'), snapshot_from_id => GETVARIABLE('avant'))
        WHERE jour = '2025-06-12') AS avant,
       (SELECT COUNT(*) FROM iceberg_scan(GETVARIABLE('tickets'), snapshot_from_id => GETVARIABLE('efface'))
        WHERE jour = '2025-06-12') AS efface,
       (SELECT COUNT(*) FROM iceberg_scan(GETVARIABLE('tickets')) WHERE jour = '2025-06-12') AS maintenant;
```

6.1 La table contre le bronze, dans les deux sens

```
-- 1. la table courante, puis ce qui manque au bronze et ce qu'il a en trop
SELECT (SELECT COUNT(*) FROM iceberg_scan(GETVARIABLE('tickets'))) AS tickets,
       (SELECT COUNT(*) FROM (SELECT * FROM read_parquet('plateforme/lac/bronze/tickets/*/*.parquet')
                              EXCEPT ALL SELECT * FROM iceberg_scan(GETVARIABLE('tickets')))) AS manquent,
       (SELECT COUNT(*) FROM (SELECT * FROM iceberg_scan(GETVARIABLE('tickets'))
                              EXCEPT ALL SELECT * FROM read_parquet('plateforme/lac/bronze/tickets/*/*.parquet'))) AS en_trop;
```


© 2026 iAlexMG. Tous droits réservés.